Curs 8

21.11.2007

Sumar

- ✓ Instrucţiuni în limbaj C
  - Simple
  - Structurate: asociative, decizionale (if, switch), ciclice (*while, do-while, for*)

7. Instrucţiunile limbajului C

Programarea calculatoarelor
Lăcrămioara Stoicu-Tivadar, 2007

Instrucţiunile limbajului C

- ✓ propoziţii prin care se specifică acţiunile care trebuie executate astfel încât să se rezolve o anumită problemă
- ✓ sintaxa:

  <eticheta:> <instrucţiune_simplă>;

  <eticheta:> <instrucţiune_compusă>;

Instrucţiunea simplă poate fi:

- ✓ instrucţiune vidă - se foloseşte atunci când este necesară o instrucţiune, dar nu se doreşte nici o acţiune: **;**
  - poate servi ca şi corp al unei instrucţiuni de ciclare *while*.
- ✓ instrucţiune de atribuire (=);
- ✓ apel de funcţie;
- ✓ instrucţiune de salt (*goto*);

Instrucţiunea structurată poate fi:

- ✓ structură asociativă
- ✓ structură selectivă
- ✓ structură ciclică

7.1. Instrucţiunile simple

Instrucţiunea de atribuire

- ✓ nume_de_variabila sau identificator de funcţie = expresie
- ✓ rezultatul executării instrucţiunii este atribuirea valorii rezultate din evaluarea expresiei, entităţii din stânga operatorului de atribuire
- ✓ *lvalue* este operandul din stânga operatorului de atribuire. Va avea valoarea rezultată din atribuirea valorii din dreapta.

Instrucţiunea de atribuire

✓ există un set de operatori de atribuire care se grupează de la dreapta la stânga

- **lvalue = expresie**
- **lvalue += expresie**
- **lvalue -= expresie**
- **lvalue *= expresie**
- **lvalue /= expresie**
- **lvalue %= expresie**
- **lvalue >>= expresie**
- **lvalue <<= expresie**
- **lvalue &= expresie**
- **lvalue v= expresie**
- **lvalue |= expresie**

Instrucţiunea de atribuire

- ✓ Atunci când se fac atribuiri trebuie să se ţină cont de tipurile entităţilor implicate;
- ✓ Dacă ambii operanzi sunt de tip aritmetic operandul drept se converteşte în tipul celui stâng înainte de asignare.
- ✓ Surse de erori:
  - tipuri ordinale submulţimi ale aceluiaşi tip

atrib1

  - atribuirea real - întreg

atrib2

  - tipuri reale diferite, tipul expresiei având un domeniu mai larg decât cel al entităţii din stânga

atrib3

atrib4

Instrucţiunea de salt

- **! nu se recomandă folosirea instrucţiunii *go to***
  - nu face parte din principiile programării structurate

- ca excepţie, se poate folosi ***go to*** atunci când în program există multe instrucţiuni imbricate de ciclare şi s-au depistat erori şi este necesar ca programul să fie întrerupt; şi pentru acest caz, există însă două proceduri predefinite care se recomandă pentru întreruperea controlată a unui program

Instrucţiunea de salt

go to eticheta

- ✓ Eticheta (nume simbolic) identifică instrucţiunea la care se va face saltul în cazul apelării instrucţiunii ***go to.***
- ✓ Atunci când este întâlnită instrucţiunea ***go to*** programul va continua cu instrucţiunea marcată prin etichetă.
- ✓ eticheta trebuie declarată anterior instrucţiunii ***go to*** în care se foloseşte

label eticheta1;

- ✓ ***label*** este cuvânt rezervat

Exemple:

```
...
label 10;
...
10: x=x+1;...
go to 10;...
...
```

```
...
label eti;
...
if a == b then goto
eti ...
eti: ...
```

```
...
label incorect;
...
go to incorect
...
if a==b then...
    incorect:...
...
```

- saltul din instrucţiunea ***if*** în afară este permis
- saltul din afară în instrucţiunea ***if*** poate determina erori în ceea ce priveşte rezultatul programului, fără a fi semnalată eroare de compilare.

Întreruperea controlată a unui program

break-continue

- ✓ *break* - determină ieşirea forţată dintr-o instrucţiune ciclică sau decizională (***while, do, for, switch***) şi continuarea execuţiei programului cu prima instrucţiune de după această instrucţiune;
- ✓ *continue* - determină saltul la sfârşitul celui mai interior ciclu ***while, do*** sau ***for*** care include apelul şi continuă cu iteraţia următoare. În cazul instrucţiunilor *while* şi *do* testarea se va executa imediat, iar în cazul instrucţiunii *for* controlul este preluat la faza de reiniţializare.

Întreruperea forţată a execuţiei unui program: de la tastatură - Ctrl – Break; Ctrl – C.

7.2. Instrucţiuni structurate

7.2.1. Structura asociativă

- în cadrul acesteia se grupează mai multe instrucţiuni rezultând astfel o secvenţă sau o instrucţiune compusă;
- instrucţiunile componente se execută secvenţial;
- blocul de instrucţiuni dintr-un program este o structură asociativă care începe cu **{** şi se termină cu **}**
- în cazul în care o secvenţă include o altă secvenţă, perechile ***{ }*** se grupează după principiul parantezelor;
- separatorul ; este obligatoriu după fiecare instrucţiune.

7.2.2. Structuri selective

- ✓ implică alegerea, în funcţie de o anumită valoare, a uneia sau mai multor variante din program

- ✓ IF

- ✓ SWITCH

Instrucţiunea IF

- ✓ Se mai numeşte structură **alternativă** sau **decizională**.
- ✓ Această structură este utilizată pentru a selecta o alternativă, din două posibile, în funcţie de o condiţie care este reprezentată printr-o **expresie** cu rezultat de **tip întreg** care poate fi
  - zero (condiţia nu este îndeplinită) sau
  - diferită de zero (condiţia este îndeplinită).

Instrucţiunea IF - sintaxa

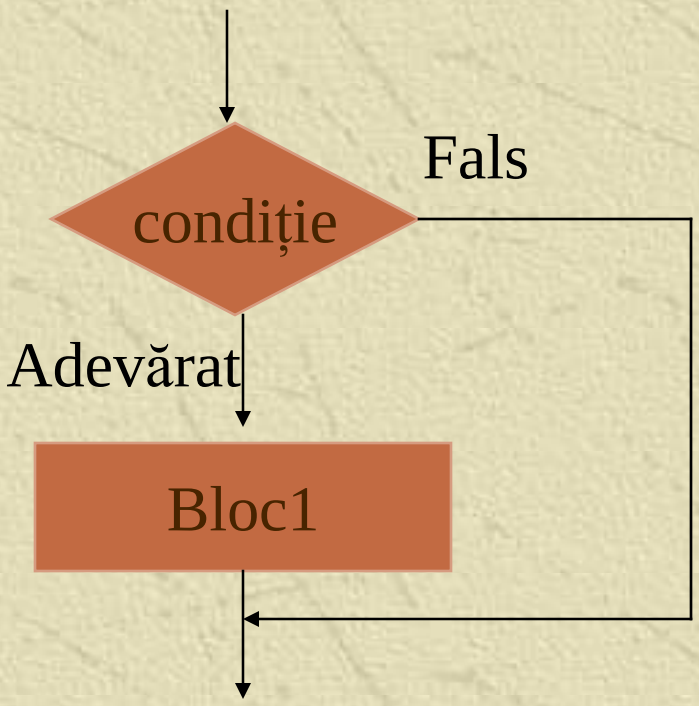
IF (conditie) Bloc1

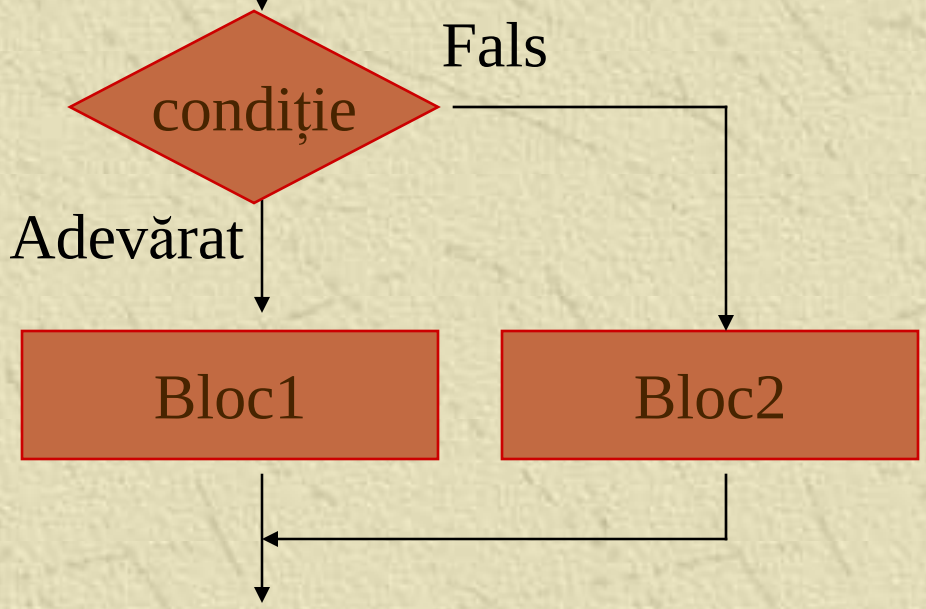
< ELSE Bloc2 >

‘Bloc1’ şi ‘Bloc2’ pot fi

- instrucţiuni simple
- structuri asociative (atunci când sunt mai multe operaţii care trebuie executate)

Instrucţiunea IF - diagrama



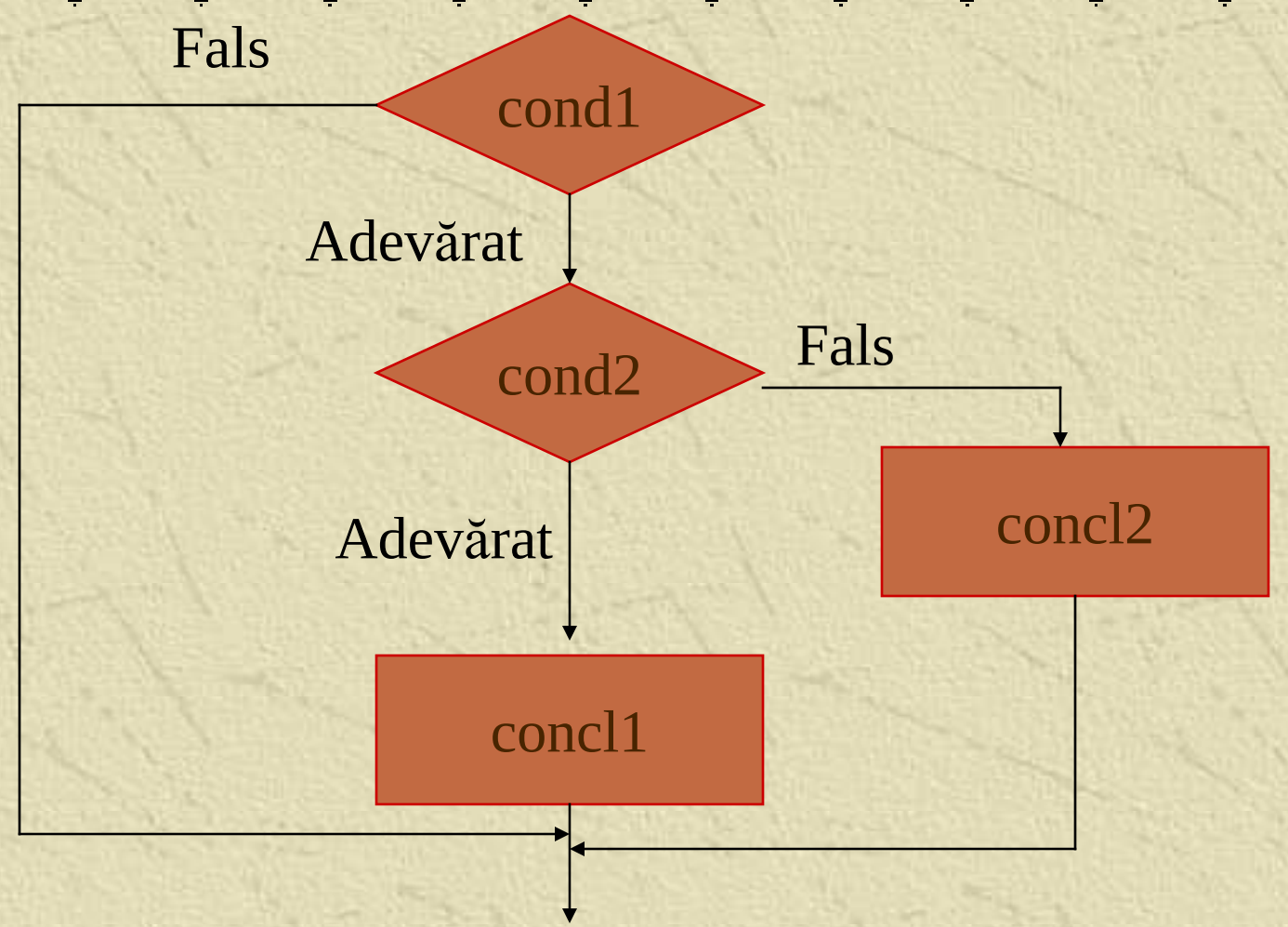


Instrucţiunea IF

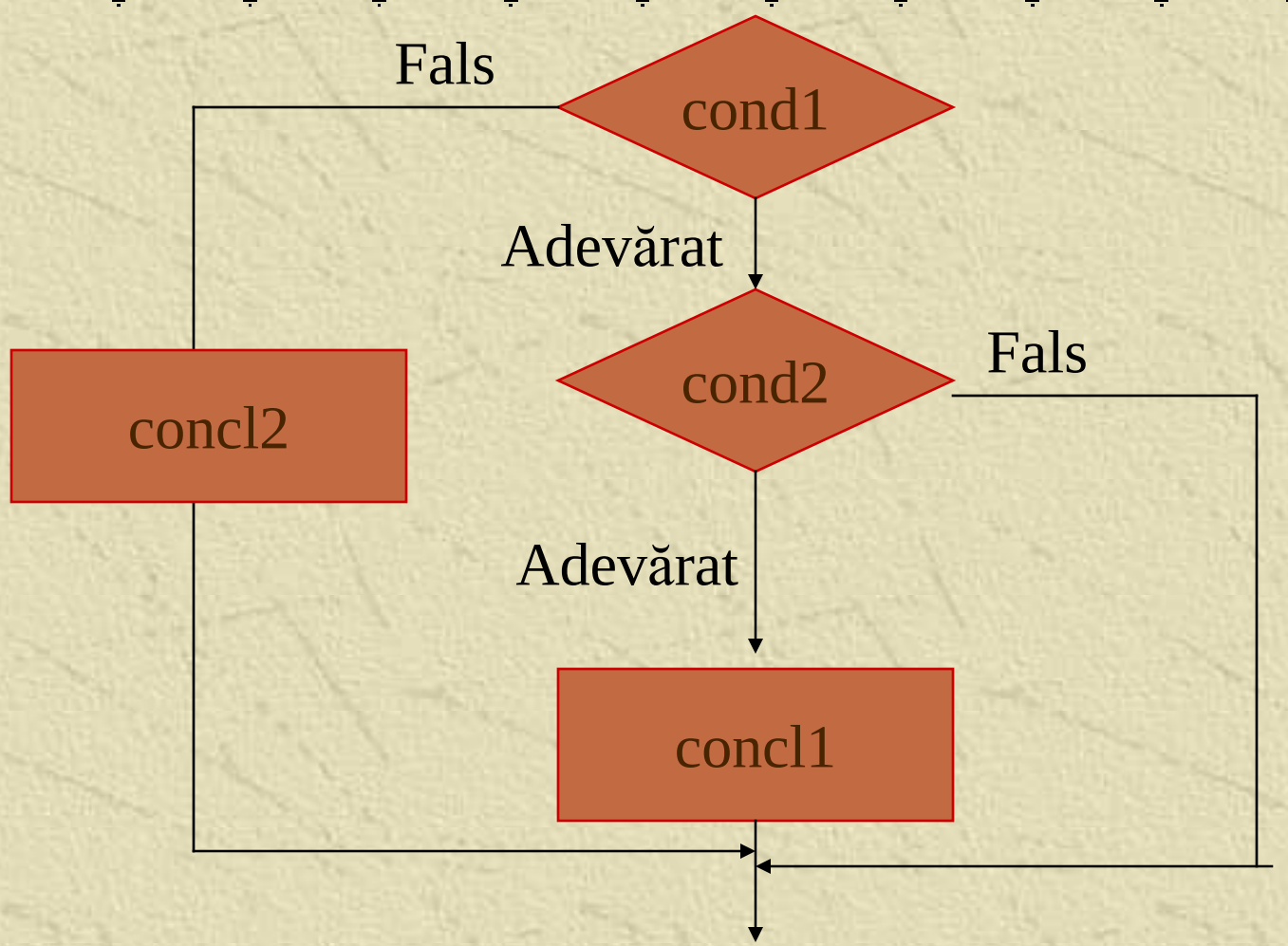
- ✓ În cazul în care există mai multe instrucţiuni ***if*** suprapuse (imbricate), ***else*** se va asocia cu cea mai apropiată instrucţiune ***if***

```
if (cond1)
   if (cond2) concl1
   else concl2
....
```

Instrucţiunea IF



Instrucţiunea IF



Instrucţiunea IF

if (cond1) { if (cond2) concl1}

else concl2

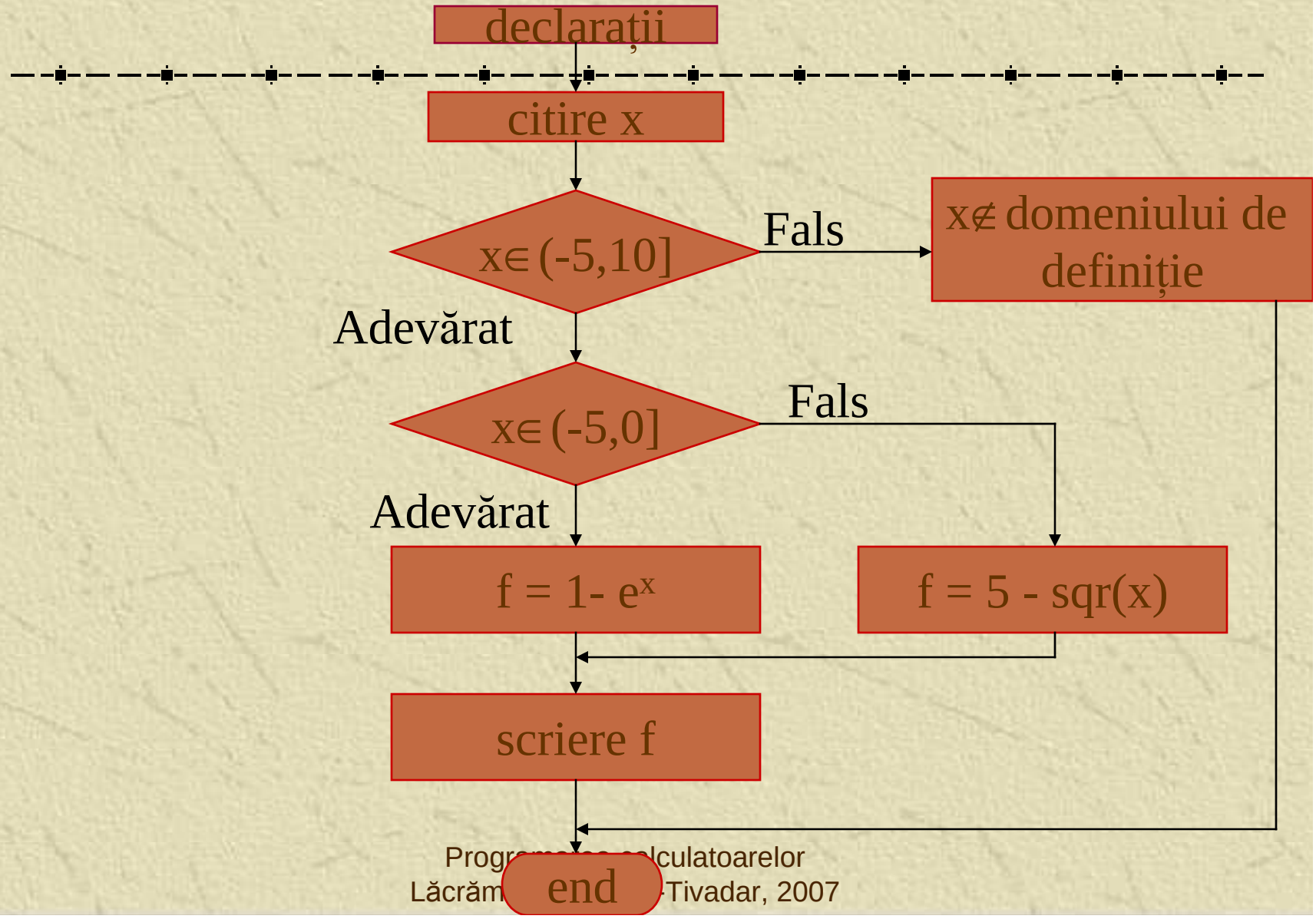
pif.c

- ✓ program care calculează valoarea unei funcţii pentru o valoare citită de la tastatură şi apoi o afişează

f:(-5,10]->R

$$f(x) = \begin{cases} 1-e^x, & x \in (-5,0] \\ 5-x^2, & x \in (0,10] \end{cases}$$

Instrucţiunea IF - diagrama programului pif.c



Instrucţiunea SWITCH - sintaxa

- ✓ **selecţie multiplă**
- ✓ se alege una dintre mai multe variante în funcţie de **valoare unei expresii**
- ✓ Sintaxa instrucţiunii **SWITCH** este:

```
SWITCH  (selector)   {
        case s1:bloc1;break;
        case s2:bloc2;break
        …
        default: bloc;
}
```

Instrucţiunea SWITCH - sintaxa

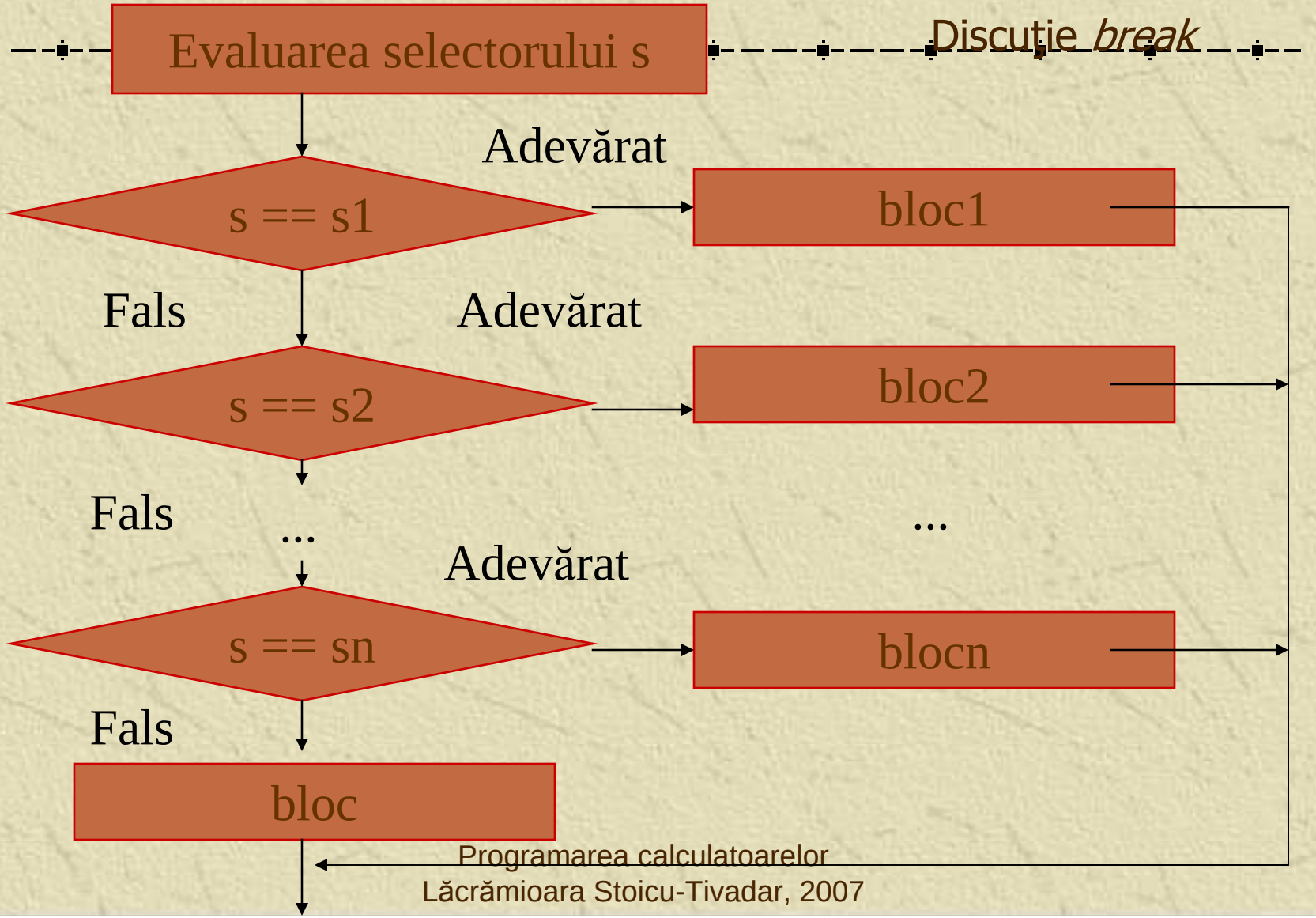
- ✓ 'selector' - expresie care reprezintă elementul decizional (deosebit de elementul decizional de la ***if***, care era o condiţie)
- ✓ s1,s2,… - constante; constantele şi selectorul trebuie să fie de acelaşi tip, iar cosntantele trebuie să fie distincte.
- ✓ bloc1,bloc2,… - instrucţiunile care se execută în funcţie de rezultatul evaluării selectorului.

Instrucţiunea SWITCH

- ✓ La o parcurgere a instrucţiunii *switch* se va executa doar o variantă din cele n (în cazul în care este precizat *break*) sau dacă nici una nu corespunde valorii selectorului există alternativa ***default.***

switch1.c

Instrucţiunea SWITCH - diagrama



Instrucţiunea SWITCH - exemple

- **SWITCH2/switch2g** - program care realizează prelucrări asupra caracterelor citite de la tastatură.
  - Se tastează câte un caracter.
  - Dacă este altceva decât cifră se semnalează eroare.
  - Dacă este 0,1,2 se neglijează, pentru restul numerelor pare se calculează pătratul, iar pentru impare cubul valorii numerice corespunzătoare şi se afişează.
  - Programul se termină dacă se tastează litera B.
- **swif.c** **swswitch.c**

7.2.3. Structuri ciclice

- implică reluarea execuţiei unui bloc de instrucţiuni în mod repetat în funcţie de o anumită **condiţie**;
- condiţia poate fi **evaluarea** unei relaţii (*adevărată* sau *falsă*) sau poate defini un anumit **domeniu de valori**
- 3 categorii de structuri ciclice
  - cu test iniţial WHILE
  - cu test final DO-WHILE
  - cu contor FOR (cu test iniţial)

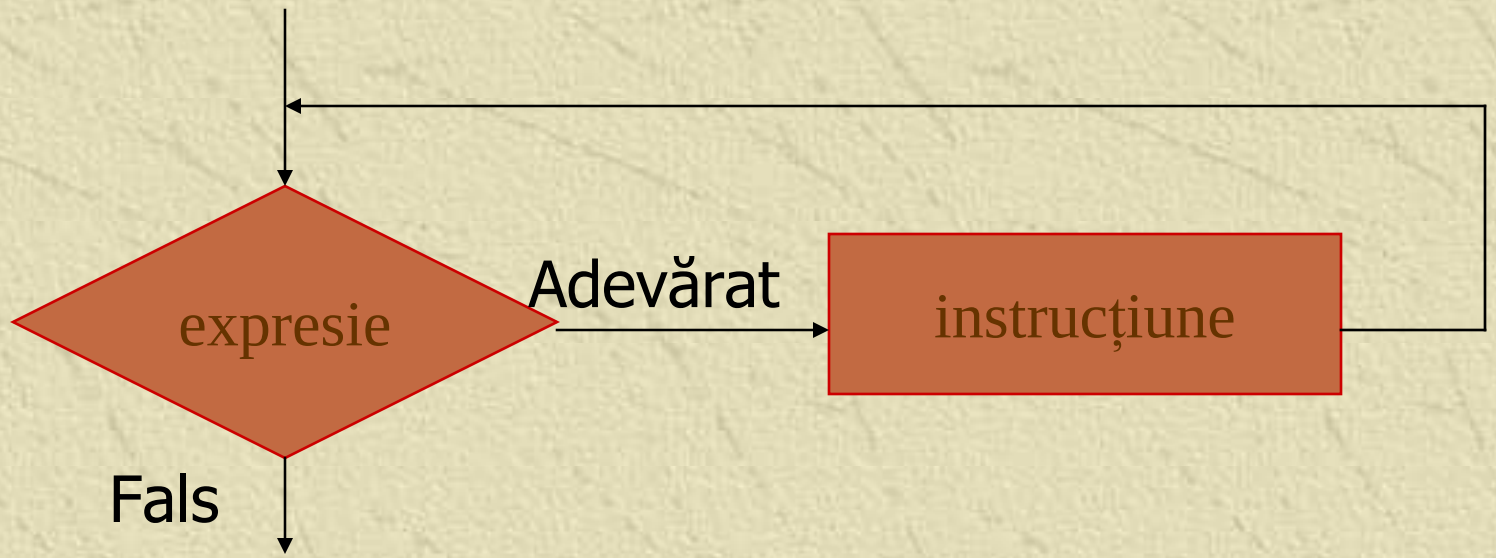
Instrucţiunea WHILE - sintaxa

WHILE (expresie) instrucţiune

- ✓ instrucţiune se va executa atât timp cât valoarea logică a expresiei este adevărată
- ✓ dacă expresia este falsă instrucţiunea nu se va executa niciodată
- ✓ pentru a ieşi din ciclare în cadrul instrucţiunii (sau în expresie) trebuie să se modifice valoarea expresiei (altfel se poate intra în cicluri infinite)

```
const n=5;…
int i,s;...
i=1;s=0;
while (i < n)  s=s+i;...
```

Instrucţiunea WHILE - diagrama



✓ **prwhile.c**

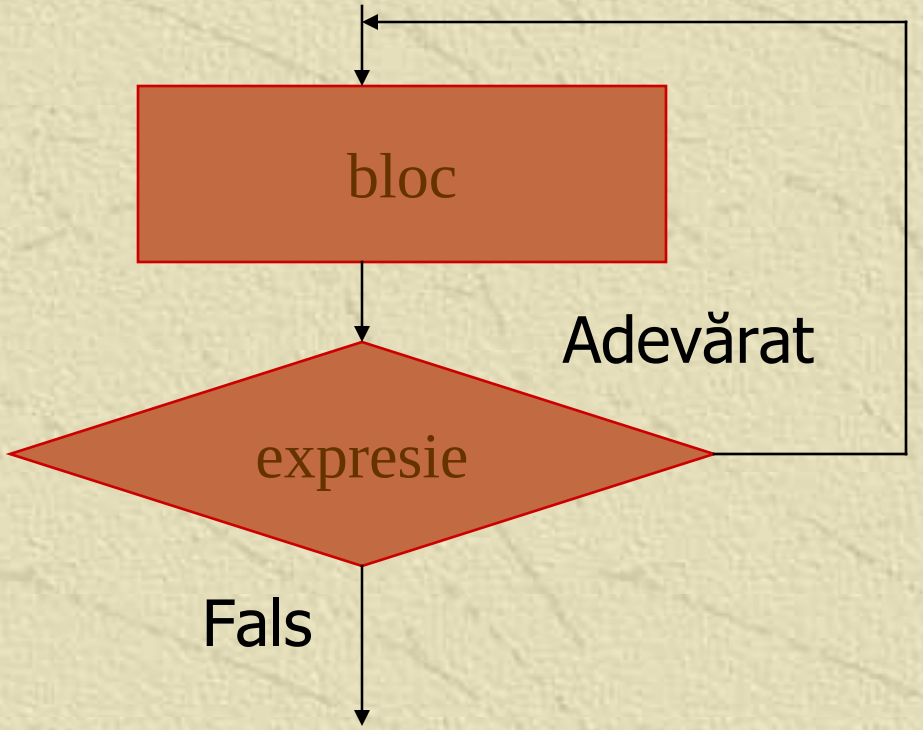
Instrucţiunea DO-WHILE - sintaxa

DO bloc WHILE (expresie)

- ✓‘bloc’ - este format dintr-una sau mai multe instrucţiuni separate prin ;
- ✓‘bloc’ va fi executat în mod repetat atât timp cât ‘expresie’ va fi adevărată
- ✓‘bloc’ se va executa cel puţin o dată

Instrucţiunea DO-WHILE - diagrama

DO bloc WHILE (expresie)



Instrucţiunea DO-WHILE - exemple

- **sumwhi.c; sumdo.c**

- dacă într-o instrucţiune ***do-while*** apare o altă instrucţiune ***do-while***, cuvintele ***do, while*** se grupează conform principiului parantezelor.

Instrucţiunea DO-WHILE - exemple

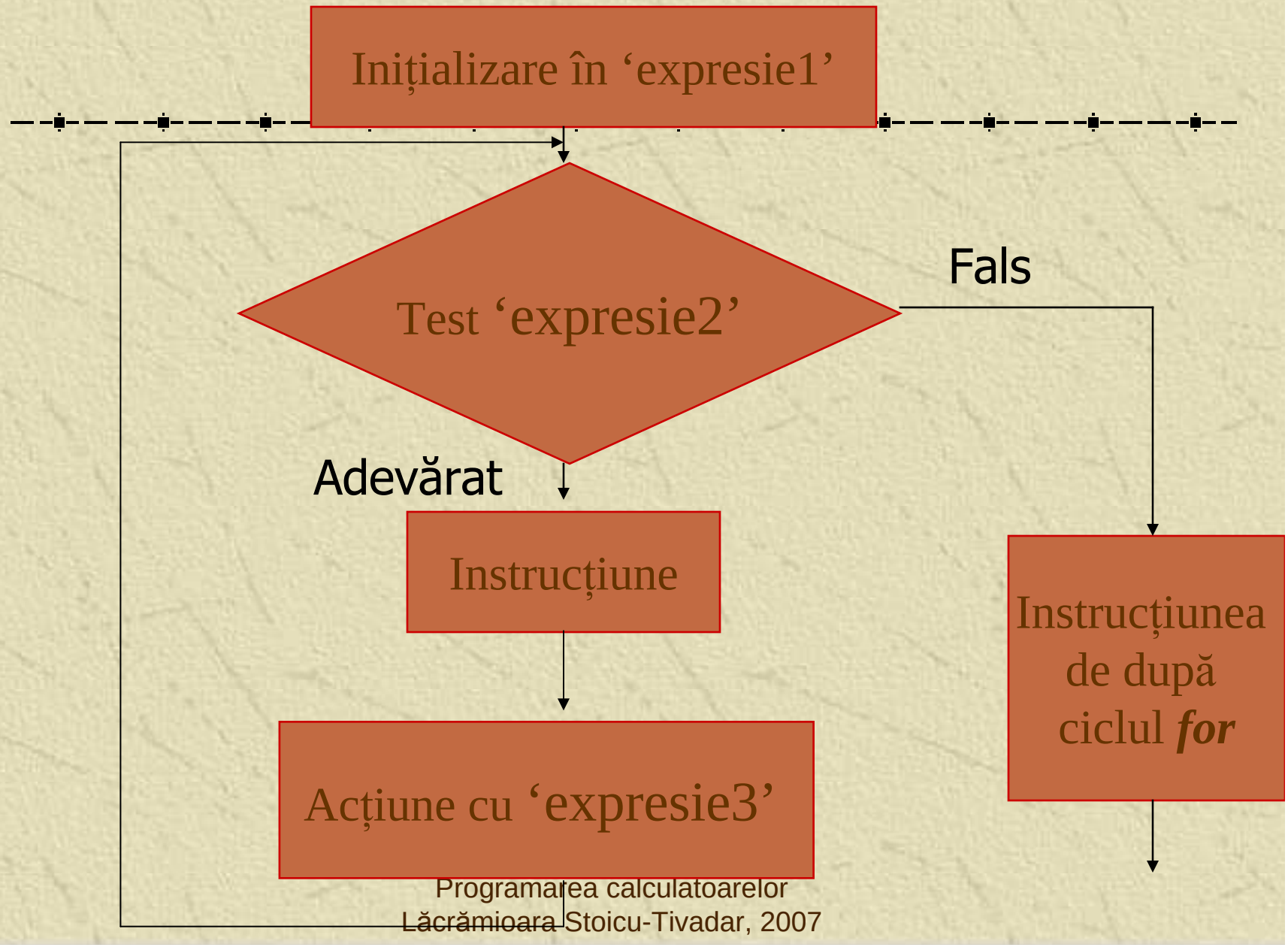
- **dowhil.c** *prelucrează un text,calculând numărul cuvintelor şi al propoziţiilor din care este alcătuit; textul se termină cu caracterul #*
- *restricţii:*
  - *cuvintele sunt alcătuite numai din litere, fiind despărţite prin orice caracter diferit de literă*
  - *propoziţiile sunt formate din cel puţin 1 cuvânt şi se termină cu punct*
  - *la începutul propoziţiilor se neglijează orice alt caracter în afară de literă sau #*

Instrucţiunea FOR - sintaxa

FOR (expresie1;expresie2;expresie3)
instrucţiune

- ✓ ‘instrucţiune’ se va executa atât timp cât condiţia precizată în ‘expresie2’ este adevărată, începând cu o valoare precizată în ‘expresie1’ cu modificări aduse în ‘expresie3’;
- ✓ testul pentru ciclul ***for*** se face iniţial, iar în cazul în care condiţia din ‘expresie2’ nu este adevărată, ciclul nu se va executa niciodată.

Instrucţiunea FOR - diagrama



Instrucţiunea FOR

- ✓ În general, ‘expresie1’ şi ‘expresie3’ sunt atribuiri sau apeluri de funcţii, iar ‘expresie2’ este o relaţie. Oricare dintre cele trei poate lipsi, dar punctul şi virgula trebuie menţinute.
- ✓ **for1.c – for3.c**

Sumar C9

- ✓ Funcţii
  - Declarare
  - Definire
  - Apel
  - Parametri formali
  - Parametri efectivi
  - Transmitere prin valoare, transmitere prin adresă.